Лекция 5. Ядро операционной системы, драйверы, процессы и службы

Цель лекции

- Понять назначение операционной системы и ядра.
- Разобрать, как приложения взаимодействуют с оборудованием через ОС.
- Понять роль системных вызовов и драйверов.
- Различать процессы и службы.
- Понимать, где администратор ищет сведения о ядре, драйверах, процессах и службах.

Список учебных вопросов

1. Назначение операционной системы как посредника между пользователем, приложениями и оборудованием.
2. Основные задачи ОС: управление ресурсами, запуск программ, работа с файлами, устройствами и сетью.
3. Ядро операционной системы: назначение и основные функции.
4. Системные вызовы: обращение приложений к возможностям ядра.
5. Драйверы устройств: связь ОС с оборудованием и типовые проблемы драйверов.
6. Процессы и службы: назначение, отличия и роль в работе серверных функций.
7. Архитектуры ядер на уровне обзора: монолитное ядро, микроядро, гибридное ядро.
8. Практическое значение для администратора.

Основные термины лекции

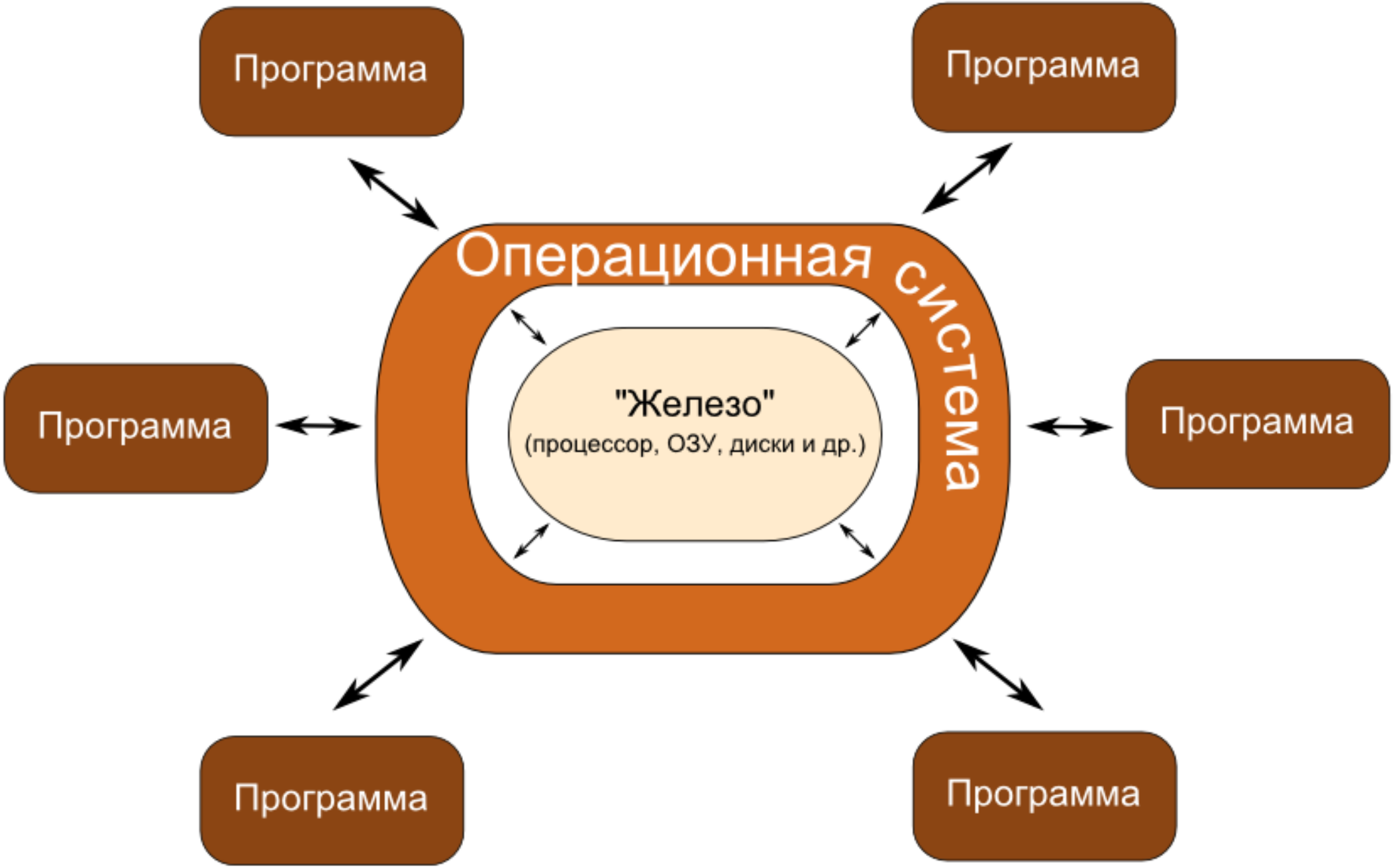
- ОС — операционная система, системное ПО для управления компьютером.
- Kernel — ядро ОС, центральная часть операционной системы.
- Driver — драйвер, программа для взаимодействия ОС с устройством.
- Process — процесс, выполняемый экземпляр программы.
- Service — служба, фоновый процесс для системной или серверной функции.
- System call — системный вызов, обращение программы к ядру ОС.

1. Операционная система как посредник

- Пользователь работает с программами, а не напрямую с железом.
- Приложения не должны напрямую управлять диском, памятью и сетью.
- ОС предоставляет контролируемый доступ к ресурсам.
- ОС защищает систему от ошибок приложений.
- ОС делает работу оборудования удобной и управляемой.

Почему нельзя работать с оборудованием напрямую

- Приложения могли бы конфликтовать за один ресурс.
- Ошибка программы могла бы повредить данные или систему.
- Нельзя было бы нормально управлять правами доступа.
- Было бы сложно поддерживать разные устройства.
- ОС вводит единые правила и контроль.


Программа
Программа
Операционная система
"Железо"
(процессор, ОЗУ, диски и др.)
Программа
Программа
Программа
Программа

2. Основные задачи ОС

- Управление процессором и временем выполнения задач.
- Управление оперативной памятью.
- Управление файлами и файловыми системами.
- Управление устройствами через драйверы.
- Управление сетью, пользователями и правами доступа.

Управление ресурсами

- ОС распределяет CPU между процессами.
- ОС выделяет RAM программам и службам.
- ОС управляет чтением и записью на диск.
- ОС контролирует доступ к сетевым интерфейсам.
- ОС фиксирует события и ошибки в журналах.

Запуск программ

- Пользователь или служба запускает программу.
- ОС создает процесс.
- Процесс получает память и доступ к разрешенным ресурсам.
- Ядро контролирует выполнение процесса.
- После завершения ОС освобождает ресурсы.

3. Ядро операционной системы

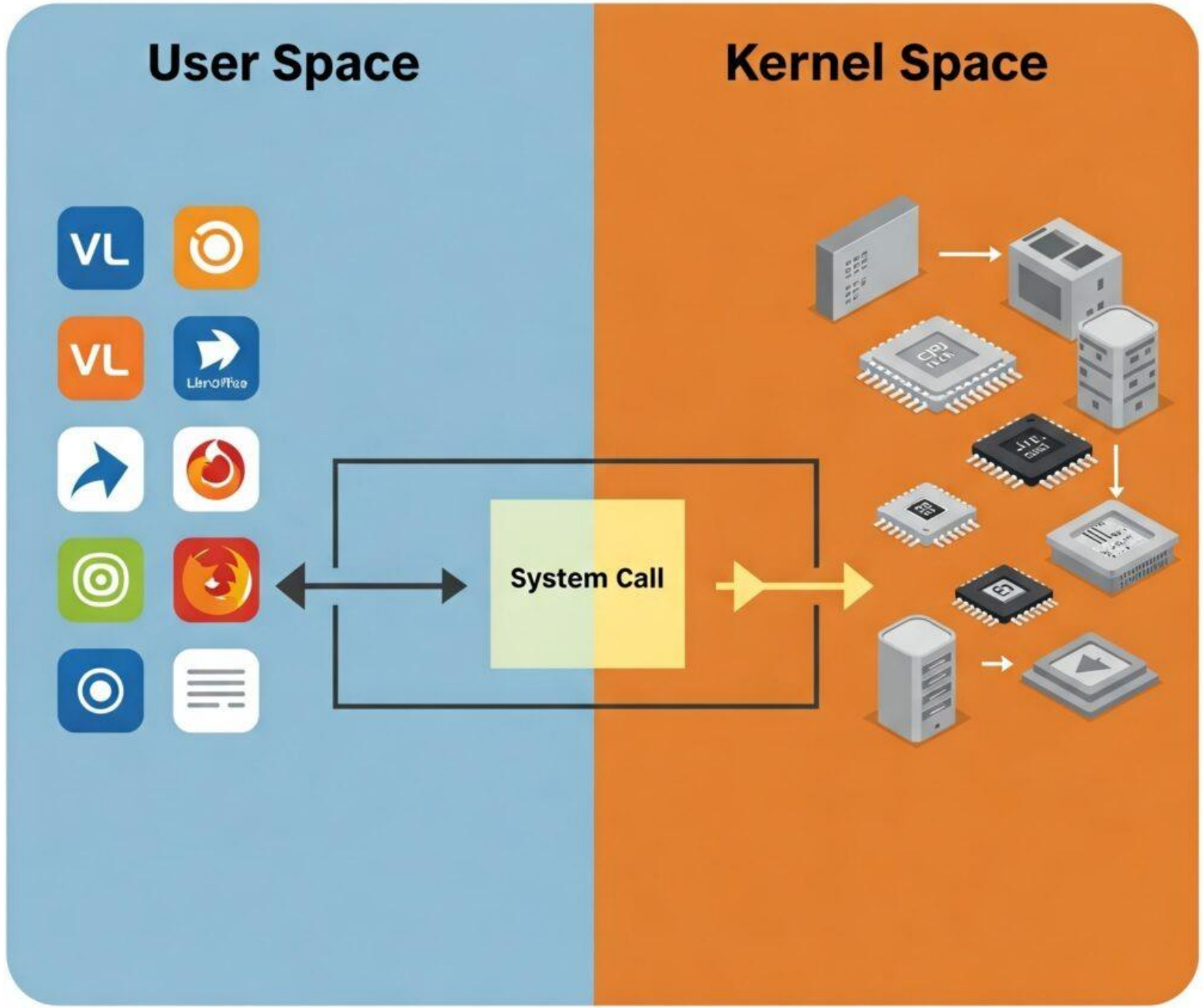
- Ядро — центральная часть ОС.
- Оно работает ближе всего к оборудованию.
- Управляет процессами, памятью, устройствами и системными вызовами.
- От стабильности ядра зависит стабильность всей ОС.
- Пользователь обычно не работает с ядром напрямую.

Что делает ядро

- Планирует выполнение процессов.
- Управляет виртуальной и физической памятью.
- Обрабатывает системные вызовы.
- Взаимодействует с драйверами устройств.
- Контролирует низкоуровневый доступ к файлам и сети.

Ядро и пользовательское пространство

- User space — область, где работают пользовательские программы.
- Kernel space — область, где работает ядро ОС.
- Разделение повышает устойчивость и безопасность.
- Программа не должна напрямую менять память ядра.
- Ошибка приложения не должна разрушать всю ОС.


User Space
Kernel Space
System Call

4. Системные вызовы

- Системный вызов — управляемое обращение программы к ядру.
- Приложение просит ядро выполнить действие с ресурсом.
- Пример: открыть файл, создать процесс, отправить пакет в сеть.
- Ядро проверяет права и выполняет операцию.
- Это безопаснее, чем прямой доступ приложения к оборудованию.

Примеры системных вызовов

- Открыть файл.
- Прочитать или записать данные.
- Создать новый процесс.
- Завершить процесс.
- Создать сетевое соединение.
- Получить сведения о времени или системе.

КАК ПРИЛОЖЕНИЕ ОБРАЩАЕТСЯ К ФАЙЛУ

Путь от функции в программе до чтения данных с диска

1 Приложение



Приложение вызывает функцию, например open("file.txt", O_RDONLY)

2 Библиотека (libc)



Вызов передаётся в библиотеку (libc), которая формирует системный вызов (syscall)

3 Ядро ОС



Ядро получает системный вызов и начинает обработку запроса

4 VFS (виртуальная файловая система)



VFS проверяет права доступа, находит нужную файловую систему и передаёт запрос ей

5 Файловая система



Файловая система находит файл (inode/ метаданные) и нужные блоки на диске

6 Драйвер устройства



Драйвер формирует запрос на чтение блоков и передаёт его контроллеру диска

7 Диск



Данные считываются с нужных секторов диска

8 Данные с диска



Секторы с данными передаются драйверу устройства

9 Драйвер устройства



Драйвер передаёт данные файловой системе

10 Драйвер устройства



Драйвер передаёт данные файловой системе

10 Файловая система



Файловая система проверяет данные и передаёт их в VFS

11 VFS



VFS передаёт данные ядру и возвращает результат вызова

12 Ядро ОС



Ядро завершает системный вызов и возвращает данные в пространство пользователя

13 Библиотека (libc)



Библиотека получает данные и возвращает их приложению (например, через read())

14 Приложение



Приложение получает данные и продолжает свою работу

Что происходит при открытии файла?

- Проверяются права доступа
- Проверяется существование файла
- Открывается (или создаётся) описание файла
- Выделяется дескриптор (число)
- Дескриптор возвращается приложению

Дескриптор файла

3 Приложение получает дескриптор (например, 3). Дальнейшие операции (read, write, close) идут через него.



Дескриптор	Описание
0	stdin (стандартный ввод)
1	stdout (стандартный вывод)
2	stderr (стандартный вывод ошибок)
3+	файлы и сокеты, открытые приложением

Уровни файлового ввода-вывода в Linux

Приложение	Ваш код (open, read, write, close)
Библиотека (libc)	Обёртки и системные вызовы
Ядро ОС	Обработка системных вызовов
VFS	Единый интерфейс для ФС
Файловая система	ext4, XFS, Btrfs и др.
Драйвер устройства	Драйвер диска (SATA/NVMe и др.)
Оборудование	Контроллер и физический диск

Итого: Приложение обращается к файлу через системные вызовы. Ядро ОС и файловая система находят данные на диске, считывают их и возвращают приложению.

5. Драйверы устройств

- Драйвер — программа, через которую ОС работает с устройством.
- Для разных устройств нужны разные драйверы.
- Драйвер скрывает детали конкретного оборудования.
- ОС через драйвер управляет диском, NIC, видеокартой, USB-устройством.
- Неверный драйвер может вызвать сбои или отсутствие устройства.

Что делает драйвер

- Определяет устройство и его возможности.
- Передает команды от ОС к устройству.
- Возвращает ОС данные и состояние устройства.
- Обрабатывает ошибки и события устройства.
- Позволяет разным программам использовать устройство через ОС.

Типовые проблемы драйверов

- Устройство не определяется.
- Устройство определяется, но работает нестабильно.
- После обновления драйвера появилась ошибка.
- Драйвер не подходит к версии ОС.
- Нет драйвера для старого или редкого оборудования.

Где смотреть драйверы

- Linux: `lsmod`, `lspci`, `lsusb`, `dmesg`.
- Windows: Device Manager.
- Windows: Event Viewer для ошибок устройств.
- Документация производителя оборудования.
- Журналы ОС после установки или обновления драйвера.

6. Процессы и службы

- Процесс — запущенный экземпляр программы.
- У процесса есть PID, память, открытые файлы и права.
- Служба — фоновый процесс для системной или серверной функции.
- Службы часто запускаются при старте ОС.
- Администратор регулярно проверяет процессы и службы.

Процесс

- Запускается пользователем, системой или другой программой.
- Использует CPU, RAM, файлы и сеть.
- Может быть интерактивным или фоновым.
- Может завершиться успешно или аварийно.
- Высокая нагрузка процесса может замедлить ОС.

Служба

- Работает в фоне без постоянного окна пользователя.
- Выполняет системную или серверную задачу.
- Примеры: web-сервер, DNS, DHCP, обновления.
- Может иметь состояние: running, stopped, failed.
- Сбой службы может нарушить работу пользователей.

ПРОЦЕСС И СЛУЖБА

Оба выполняют задачи в системе, но работают по-разному и имеют разное назначение.

ПРОЦЕСС — это экземпляр выполняемой программы в оперативной памяти.

СЛУЖБА — это фоновый процесс, который работает без взаимодействия с пользователем и управляется системой.

ПРОЦЕСС



Запуск приложения пользователем (или системой) → Операционная система создаёт процесс → Процесс получает ресурсы: CPU, память, файлы, сокеты и др. → Процесс выполняет задачу

Завершение процесса (нормальное или по ошибке)

Характеристики процесса

- Имеет свой адресное пространство
- Выполняется от имени пользователя или системы
- Имеет родительский процесс (кроме системных)
- Завершается после выполнения задачи
- Может запускать дочерние процессы
- Виден в диспетчере задач

Примеры процессов

- Браузер (chrome.exe)
- Блокнот (notepad.exe)
- Командная строка (cmd.exe)
- Редактор кода (code.exe)
- ...

СРАВНЕНИЕ

Назначение
Процесс — для выполнения конкретной задачи.
Служба — для постоянной фоновой работы.

Запуск
Процесс — вручную пользователем или другим процессом.
Служба — системой (автоматически).

Время работы
Процесс — пока работает задача.
Служба — долго или постоянно.

Взаимодействие с пользователем
Процесс — может иметь интерфейс.
Служба — не имеет интерфейса.

Надёжность
Процесс — может завершиться с ошибкой.
Служба — может быть настроена на автоматический перезапуск.

СЛУЖБА



Система запускает службу (при загрузке или по расписанию) → Служба регистрируется в системе служб → Служба работает в фоне, без взаимодействия с пользователем → Служба продолжает работать до остановки или выключения системы

Автоматический перезапуск (если настроено)

Характеристики службы

- Работает в фоновом режиме
- Запускается автоматически системой
- Не имеет интерфейса пользователя
- Может работать под отдельной учётной записью
- Не зависит от пользовательского сеанса
- Может быть настроена на автоматический перезапуск
- Управляется через службы ОС

Примеры служб

- Windows Update (wuauserv)
- Брандмауэр (MpsSvc)
- Диспетчер печати (Spooler)
- Служба баз данных (MySQL)
- ...

ГДЕ ПОСМОТРЕТЬ?

Windows

- Процессы — Диспетчер задач (Ctrl+Shift+Esc) вкладка «Процессы»
- Службы — services.msc

Linux

- Процессы — ps aux, top, htop
- Службы — systemctl list-units --type=service

КАК УПРАВЛЯТЬ СЛУЖБОЙ?

Windows (команды в CMD/PowerShell от имени администратора):

- sc start <имя_службы>
- sc stop <имя_службы>
- sc config <имя_службы> start= auto
- sc query <имя_службы>

Linux (systemctl):

- systemctl start <служба>
- systemctl stop <служба>
- systemctl enable <служба>
- systemctl status <служба>

ИТОГО Процесс — это временное выполнение задачи. Служба — это постоянная фоновая работа, управляемая системой.

7. Архитектуры ядер

- Архитектура ядра описывает, как организованы компоненты ОС.
- Сейчас важно понимать идею, а не детали реализации.
- Основные подходы: монолитное ядро, микроядро, гибридное ядро.
- Разные ОС используют разные инженерные компромиссы.
- Архитектура влияет на производительность, надежность и расширяемость.

Монолитное ядро

- Большая часть функций работает в пространстве ядра.
- Обычно высокая производительность.
- Ошибка в компоненте ядра может серьезно повлиять на систему.
- Linux относится к монолитным ядрам с поддержкой модулей.
- Модули позволяют добавлять драйверы и функции без полной пересборки ядра.

Микроядро

- В ядре остается минимальный набор функций.
- Многие службы выносятся в пользовательское пространство.
- Идея: повысить изоляцию и надежность.
- Может быть сложнее в реализации.
- В чистом виде встречается реже в массовых ОС.

Гибридное ядро

- Совмещает идеи монолитного ядра и микроядра.
- Часть служб работает ближе к ядру ради производительности.
- Часть архитектурных идей направлена на модульность.
- Windows часто описывают как систему с гибридным ядром.
- Для администратора важнее понимать последствия: драйверы, службы, стабильность.

АРХИТЕКТУРЫ ЯДЕР ОС

Разные подходы к организации взаимодействия компонентов операционной системы

Производительность — эффективность и скорость

Надёжность — изоляция и отказоустойчивость

Расширяемость — лёгкость добавления новых компонентов

Сложность — реализация и сопровождение

1. Монолитное ядро

Все основные компоненты работают в пространстве ядра и имеют прямой доступ к оборудованию.



- ✓ Высокая производительность
- ✓ Простой обмен данными между компонентами
- ✗ Сложно расширять и модифицировать
- ✗ Ошибки в ядре могут привести к сбою всей системы

Примеры: Linux, *BSD, Solaris, MS-DOS

2. Модульное ядро

Базовое ядро содержит минимум функций, остальные компоненты загружаются как модули.



- ✓ Гибкость и расширяемость
- ✓ Модули можно загружать и выгружать без перезагрузки
- ✗ Сложнее монолитного ядра
- ✗ Небольшие накладные расходы

Примеры: Linux (модули), Solaris, FreeBSD

3. Микроядро

В ядре остаются только базовые функции: планирование, управление памятью и IPC. Остальные службы в пространстве пользователя.



- ✓ Высокая надёжность (изоляция сервисов)
- ✓ Легче разрабатывать и тестировать
- ✗ Ниже производительность
- ✗ Сложный обмен данными (IPC)

Примеры: MINIX, QNX, seL4, Mach

4. Гибридное ядро

Комбинирует подходы монолитного ядра и микроядра: многие сервисы работают в ядре, но с модульной организацией.



- ✓ Баланс производительности и надёжности
- ✓ Широко используется на практике
- ✗ Более сложное ядро
- ✗ Больше кода в пространстве ядра

Примеры: Windows NT, macOS (XNU), Android (Linux + компоненты)

5. Экзоядро

Ядро предоставляет только механизм безопасного разделения ресурсов. Политики управления реализуются в пользовательских библиотеках.



- ✓ Максимальная гибкость
- ✓ Можно адаптировать ОС под задачи
- ✗ Очень сложная реализация
- ✗ Практически не используется

Примеры: MIT Exokernel (исследовательский проект)

Итого

Выбор архитектуры — это компромисс между производительностью, надёжностью, гибкостью и сложностью разработки. Большинство современных ОС используют гибридные или модульные архитектуры.

8. Практическое значение для администратора

- Понимание ядра помогает читать сообщения ОС.
- Понимание драйверов помогает диагностировать оборудование.
- Понимание процессов помогает искать перегрузку.
- Понимание служб помогает восстанавливать серверные функции.
- Журналы связывают симптомы с причинами.

Что смотреть в Linux

- `uname -a`: сведения о ядре.
- `lsmod`: загруженные модули ядра.
- `ps aux`: список процессов.
- `top` или `htop`: нагрузка процессов.
- `systemctl status`: состояние службы.
- `journalctl`: журналы systemd и служб.

Что смотреть в Windows

- Device Manager: устройства и драйверы.
- Task Manager: процессы и базовая нагрузка.
- Services: службы и их состояние.
- Event Viewer: события ОС и приложений.
- System Information: сведения о системе.
- PowerShell: автоматизированный сбор информации.

Итоги лекции

- ОС управляет ресурсами и защищает оборудование от прямого доступа приложений.
- Ядро является центральной частью ОС.
- Системные вызовы дают программам контролируемый доступ к функциям ядра.
- Драйверы связывают ОС с устройствами.
- Процессы и службы — ключевые объекты диагностики администратора.

Контрольные вопросы

1. Какие задачи выполняет операционная система?
2. Какую роль выполняет ядро ОС?
3. Почему пользовательские программы не должны работать с оборудованием напрямую?
4. Для чего используются системные вызовы?
5. Какую функцию выполняет драйвер устройства?
6. Чем процесс отличается от службы?
7. Почему серверные функции часто работают как службы?
8. Как драйвер, процесс или служба могут стать причиной сбоя системы?